

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CO3084 - Mật mã học và Mã hóa Thông tin (TN)

Problem Set 2

Symmetric Cryptography

SVTH: Nguyễn Tấn Tài - 2212990

TP. Hồ Chí Minh, Tháng 10/2025

Mục lục

1	Tính Giả Ngẫu Nhiên (Pseudorandomness)	1
1.1	Câu 1. (10 điểm) Bộ Sinh Giả Ngẫu Nhiên (Pseudorandom Generators - PRGs)	1
1.1.1	(a) (5 điểm) Giải thích các hạn chế của mã hóa một lần (one-time pad) đối với việc mã hóa thực tế và tại sao các bộ sinh giả ngẫu nhiên (PRGs) lại cần thiết trong các hệ thống mật mã hiện đại	1
1.1.2	(b) (5 điểm) Phân tích các tác động bảo mật của cấu trúc PRG sau đây, trong đó G là một PRG an toàn	1
1.2	Câu 2. (10 điểm) Hàm Giả Ngẫu Nhiên và Hoán Vị (Pseudorandom Functions and Permutations) .	2
1.2.1	(a) (5 điểm) Phân tích cấu trúc PRF $F(K, X) = G(K) \oplus X$	2
1.2.2	(b) (5 điểm) So sánh PRFs và PRPs	2
2	Tấn Công Văn Bản Rõ Đã Chọn và Văn Bản Mã Hóa Đã Chọn	4
2.1	Câu 1. (10 điểm) Bảo mật CPA (Chosen-Plaintext Attack)	4
2.1.1	(a) (5 điểm) Định nghĩa và vì sao mã hóa xác định thất bại CPA	4
2.1.2	(b) (5 điểm) Đánh giá CPA cho các lược đồ cụ thể	4
2.2	Câu 2. (10 điểm) Bảo mật CCA và Mã hóa Xác thực	5
2.2.1	(a) (3 điểm) Tấn công format-oracle (null-oracle) trên CTR	5
2.2.2	(b) (4 điểm) Đánh giá CCA và AE cho các cấu trúc	5
2.2.3	(c) (3 điểm) AES-GCM (Galois/Counter Mode)	5
3	Hàm Băm Chống Va Chạm (Collision-Resistant Hash Functions)	7
3.1	Câu 1. (15 điểm) Thuộc tính Hàm băm	7
3.1.1	(a) (5 điểm) Khả năng chống va chạm (Collision Resistance)	7
3.1.2	(b) (5 điểm) Cấu trúc hàm băm	7
3.1.3	(c) (5 điểm) Sự phát triển của hàm băm	8
3.2	Câu 2. (15 điểm) Băm mật khẩu (Password Hashing)	8
3.2.1	(a) (5 điểm) Phân tích lưu trữ mật khẩu	8
3.2.2	(b) (5 điểm) Muối (Salting)	9
3.2.3	(c) (5 điểm) Hàm băm mật khẩu chuyên dụng	9
4	Nghiên Cứu Điển Hình Mật Mã Ứng Dụng	10
4.1	Câu 1. (10 điểm) Phân Tích Các Chế Độ Mã Khối	10
4.1.1	(a) (4 điểm) Ứng dụng lưu trữ tệp an toàn (Tệp lớn)	10
4.1.2	(b) (3 điểm) Ứng dụng nhắn tin thời gian thực (Tin nhắn ngắn)	10
4.1.3	(c) (3 điểm) Cụ thể đối với AES-GCM	11
4.2	Câu 2. (10 điểm) Phân Tích Bảo Mật Hàm Băm	11
4.2.1	(a) Xác định ít nhất ba lỗ hổng bảo mật	12
4.2.2	(b) Kịch bản tấn công cụ thể	12
4.2.3	(c) Đề xuất cải tiến	12
4.2.4	(d) Thiết kế hệ thống thay thế an toàn	12
4.3	Câu 3. (10 điểm) Thiết Kế Hệ Thống Quản Lý Mật Khẩu	13
4.3.1	(a) Nguyên thủy mật mã để lưu trữ mật khẩu	13
4.3.2	(b) Cơ chế khôi phục mật khẩu	13
4.3.3	(c) Cân bằng Bảo mật vs Hiệu suất	14
4.3.4	(d) Lỗ hổng tiềm ẩn và giảm thiểu	14

1 Tính Giả Ngẫu Nhiên (Pseudorandomness)

1.1 Câu 1. (10 điểm) Bộ Sinh Giả Ngẫu Nhiên (Pseudorandom Generators - PRGs)

1.1.1 (a) (5 điểm) Giải thích các hạn chế của mã hóa một lần (one-time pad) đối với việc mã hóa thực tế và tại sao các bộ sinh giả ngẫu nhiên (PRGs) lại cần thiết trong các hệ thống mật mã hiện đại

Trả lời:

Hạn chế của Mã Một Lần (OTP): OTP cung cấp tính bí mật hoàn hảo (perfect secrecy). Tuy nhiên, hạn chế cốt lõi là khóa (K) phải dài ít nhất bằng văn bản gốc (M). Điều này tạo ra một nghịch lý hậu cần ("trứng có trước hay gà có trước") vì để truyền thông tin mật (n bit), người dùng đã phải chia sẻ n bit khóa an toàn trước đó.

Sự cần thiết của PRGs: Mật mã học thực tế cần các lược đồ mà khóa (k_s) có thể nhỏ hơn nhiều so với thông điệp ($|k_s| < |m|$). Bộ Sinh Giả Ngẫu Nhiên (PRG) là giải pháp: nó nhận một seed ngắn (S) và mở rộng nó thành một luồng khóa dài hơn $G(S)$. Mặc dù đầu ra của PRG không thể đạt được phân phối đồng nhất thực sự (do số lượng đầu vào nhỏ hơn không gian đầu ra), nó vẫn được coi là an toàn nếu đầu ra đó không thể phân biệt được về mặt tính toán (computationally indistinguishable) với một chuỗi ngẫu nhiên thực sự.

1.1.2 (b) (5 điểm) Phân tích các tác động bảo mật của cấu trúc PRG sau đây, trong đó G là một PRG an toàn

$$H(S) = A \| B \| C \| D \text{ trong đó } A \| B = G(S) \text{ và } C \| D = G(B)$$

Xác định xem H có phải là một PRG an toàn hay không. Nếu không, hãy cung cấp một bộ phân biệt (distinguisher) có thể phân biệt $H(S)$ với một chuỗi thực sự ngẫu nhiên với lợi thế không đáng kể (non-negligible advantage).

Trả lời:

1) Cấu trúc PRG nối tiếp $H(S)$.

$$H(S) = A \| B \| C \| D, \quad A \| B = G(S), \quad C \| D = G(B)$$

Trong đó G là một PRG an toàn. Điểm yếu cốt lõi nằm ở việc tái sử dụng nửa sau B làm seed mới cho lần mở rộng tiếp theo.

2) Ý tưởng chính: Mỗi liên hệ nội tại.

Nếu $H(S)$ là chuỗi ngẫu nhiên thực sự thì bốn phần A, B, C, D sẽ độc lập. Tuy nhiên, trong $H(S)$ thật ta có ràng buộc tất định $C \| D = G(B)$, tạo nên mối liên hệ kiểm chứng được giữa nửa trước và nửa sau.

3) Bộ phân biệt D hoạt động như thế nào.

Khi nhận chuỗi thử thách $Y = Y_1 \| Y_2 \| Y_3 \| Y_4$, bộ phân biệt D thực hiện:

Bước	Hành động	Mục đích
1	Lấy Y_2 (ứng với B)	Xác định seed thứ hai
2	Tính $G(Y_2)$	Dự đoán giá trị $Y_3 \ Y_4$ nếu là thật
3	So sánh $G(Y_2) \stackrel{?}{=} Y_3 \ Y_4$	Kiểm tra quan hệ nội tại
4	Khớp: đoán REAL; ngược lại: RAND	Phân biệt nguồn chuỗi

4) Phân tích xác suất và lợi thế.

Trường hợp chuỗi thật (L_{real}^H): quan hệ $G(B) = C \| D$ luôn đúng nên $\Pr[D \text{ đoán đúng}] = 1$.

Trường hợp chuỗi ngẫu nhiên (L_{rand}^H): Y_2, Y_3, Y_4 độc lập, do đó

$$\Pr[G(Y_2) = Y_3 \| Y_4] = \frac{1}{2^{2\lambda}}.$$

5) Kết luận lợi thế.

$$\text{Adv}(D) = 1 - \frac{1}{2^{2\lambda}} \approx 1.$$

Lợi thế xấp xỉ 1 (*không thể bỏ qua, non-negligible*), do đó D phân biệt dễ dàng giữa $H(S)$ thật và chuỗi ngẫu nhiên. Suy ra $H(S)$ **không** phải là một PRG an toàn.

Tóm tắt một câu: Cấu trúc $H(S)$ thất bại vì đầu ra sau phụ thuộc có thể kiểm chứng vào đầu ra trước ($C||D = G(B)$), khiến bộ phân biệt dễ dàng nhận biết chuỗi thật so với chuỗi ngẫu nhiên.

1.2 Câu 2. (10 điểm) Hàm Giả Ngẫu Nhiên và Hoán Vị (Pseudorandom Functions and Permutations)

1.2.1 (a) (5 điểm) Phân tích cấu trúc PRF $F(K, X) = G(K) \oplus X$

Yêu cầu: Xét cấu trúc PRF $F(K, X) = G(K) \oplus X$, trong đó G là một PRG an toàn. Xác định F có phải là một PRF an toàn không. Nếu không, mô tả một bộ phân biệt có thể phân biệt hiệu quả F với một hàm ngẫu nhiên.

Trả lời:

$F(K, X) = G(K) \oplus X$ **không** phải là một PRF an toàn vì tồn tại quan hệ tuyến tính loại bỏ $G(K)$ khi so sánh hai truy vấn trên cùng một khóa.

Xét bộ phân biệt D :

- Truy vấn *oracle* với hai đầu vào tùy ý X_1, X_2 để nhận $Y_1 = F(K, X_1)$ và $Y_2 = F(K, X_2)$.
- Tính và kiểm tra $Y_1 \oplus Y_2 \stackrel{?}{=} X_1 \oplus X_2$.
- Nếu đúng thì đoán REAL; ngược lại đoán RAND.

Phân tích lợi thế:

$$Y_1 \oplus Y_2 = (G(K) \oplus X_1) \oplus (G(K) \oplus X_2) = X_1 \oplus X_2$$

Trong thư viện $L_{prf-real}^F$ đẳng thức luôn đúng (xác suất 1). Còn trong $L_{prf-rand}^F$, Y_1, Y_2 độc lập nên $\Pr[Y_1 \oplus Y_2 = X_1 \oplus X_2] = 2^{-m}$ với m là độ dài đầu ra. Do đó $\text{Adv}(D) \approx 1$.

1.2.2 (b) (5 điểm) So sánh PRFs và PRPs

Yêu cầu:

- Nêu khác biệt chính trong định nghĩa và thuộc tính.
- Mô tả cách PRPs có thể được "hạ cấp" thành PRFs, nhưng không nhất thiết ngược lại.
- Giải thích vì sao va chạm là không thể tránh khỏi với PRFs nhưng không phải với PRPs.

Trả lời:

Thuộc tính	PRF (Hàm giả ngẫu nhiên)	PRP (Hoán vị giả ngẫu nhiên)
Định nghĩa	Hàm có khóa, đầu ra không phân biệt được với hàm ngẫu nhiên lý tưởng	Hoán vị có khóa (song ánh), không phân biệt được với hoán vị ngẫu nhiên lý tưởng
Khả nghịch	Không yêu cầu nghịch đảo hiệu quả	Bắt buộc tồn tại nghịch đảo F^{-1} hiệu quả
Va chạm đầu ra	Có thể xảy ra (do nguyên lý sinh nhật)	Không xảy ra (song ánh)
Truy vấn	Truy vấn chọn lọc tới <i>oracle</i>	Truy vấn chọn lọc tới <i>oracle</i> (cả F và thường cả F^{-1})

Suy biến ($PRP \Rightarrow PRF$): Mọi PRP cũng là PRF khi chỉ xét tính giả ngẫu nhiên (bỏ qua khả nghịch), nhưng không phải mọi PRF đều là PRP vì PRF không đảm bảo tính song ánh/khả nghịch.

Va chạm: PRF mô phỏng hàm ngẫu nhiên nên va chạm là không tránh khỏi khi ánh xạ từ không gian đầu vào lớn sang không gian đầu ra cố định (nguyên lý lỗ bồ câu); PRP là song ánh nên không có va chạm.

2 Tấn Công Văn Bản Rõ Đã Chọn và Văn Bản Mã Hóa Đã Chọn

2.1 Câu 1. (10 điểm) Bảo mật CPA (Chosen-Plaintext Attack)

2.1.1 (a) (5 điểm) Định nghĩa và vì sao mã hóa xác định thất bại CPA

Yêu cầu:

```
L_Sigma^{cpa-real}      L_Sigma^{cpa-rand}
K <- Sigma.K            cpa.enc(M) :
cpa.enc(M) :            C <- Sigma.C(|M|)
    C := Sigma.Enc(K, M)    return C
    return C
```

- 1) Giải thích vì sao lược đồ mã hóa *xác định* luôn thất bại với CPA.
- 2) Xây dựng một bộ phân biệt đơn giản phá vỡ CPA cho bất kỳ lược đồ xác định nào.
- 3) Nêu các lỗ hổng thực tế khi dùng mã hóa không an toàn CPA.

Trả lời:

Mã hóa xác định thất bại CPA. Với cùng thông điệp M , lược đồ xác định luôn cho cùng ciphertext C . Kẻ tấn công có thể nhận biết lặp lại và xây dựng “từ điển”.

Bộ phân biệt D .

1. Truy vấn *oracle* mã hóa: $C_1 \leftarrow \text{cpa.enc}(M_0)$.
2. Truy vấn lần hai: $C_2 \leftarrow \text{cpa.enc}(M_0)$.
3. Nếu $C_1 = C_2$ đoán REAL; ngược lại RAND.

Lợi thế $\text{Adv}(D) = 1$ trong lược đồ xác định, vì $C_1 = C_2$ luôn đúng.

Lỗ hổng thực tế.

- Rò rỉ lặp lại: nhận biết mẫu và xây “từ điển” plaintext đã biết.
- Rò rỉ độ dài: độ dài vẫn rò rỉ ngay cả với CPA-secure; các mẫu độ dài có thể tiết lộ thông tin phụ.

2.1.2 (b) (5 điểm) Đánh giá CPA cho các lược đồ cụ thể

Yêu cầu: Với mỗi lược đồ sau, cho biết có CPA-secure không; nếu không, chỉ ra tấn công.

1. $\text{Enc}(K, M) = (R, F(K, R) \oplus M)$, $R \leftarrow \{0, 1\}^\lambda$, F là PRF an toàn.
2. $\text{Enc}(K, M) = (R, F(K, M) \oplus R)$, $R \leftarrow \{0, 1\}^\lambda$, F là PRF an toàn.
3. AES chế độ ECB.
4. AES chế độ CTR với IV ngẫu nhiên.

Trả lời:

- 1) **CPA-secure: CÓ.** Hoạt động như mật mã dòng với nonce/IV R ngẫu nhiên; keystream $F(K, R)$ khác nhau mỗi lần.
- 2) **CPA-secure: KHÔNG.** Lược đồ xác định theo M vì $F(K, M)$ cố định. Truy vấn hai lần cùng M_0 được (R_1, C_1) , (R_2, C_2) và

$$C_1 \oplus C_2 = (F(K, M_0) \oplus R_1) \oplus (F(K, M_0) \oplus R_2) = R_1 \oplus R_2.$$

Quan hệ tuyến tính dễ kiểm chứng.

- 3) **CPA-secure: KHÔNG.** ECB là xác định: khối $M_i = M_j \Rightarrow C_i = C_j$.
- 4) **CPA-secure: CÓ.** CTR biến PRP thành PRF với IV ngẫu nhiên và bộ đếm; đầu vào mỗi khối là duy nhất, đảm bảo CPA.

2.2 Câu 2. (10 điểm) Bảo mật CCA và Mã hóa Xác thực

2.2.1 (a) (3 điểm) Tấn công format-oracle (null-oracle) trên CTR

Yêu cầu:

1. Giải thích cách tấn công null-oracle hoạt động chống CTR và vì sao vẫn phá hoại dù CTR an toàn CPA.
2. Nêu một kịch bản thực tế nơi format-oracle có thể bị lộ.
3. Tính số truy vấn gần đúng để khôi phục tệp 1 KB bằng null-oracle và giải thích tính khả thi.

Trả lời:

Cách hoạt động (khai thác tính dẻo). Chọn một byte thử $g \in \{01, \dots, FF\}$, lật bit tương ứng trong ciphertext để tạo C' , gửi C' đến oracle giải mã. Nếu byte giải mã $M'_i \oplus g = 00$ thì oracle rò rỉ tín hiệu (ví dụ lỗi/crash/boolean), suy ra $M_i = g$.

Vì sao tàn phá. CTR an toàn CPA nhưng dẻo (malleable) do XOR trên keystream; format-oracle biến rò rỉ nhỏ thành kênh thông tin từng byte.

Kịch bản thực tế (padding oracle). Trong CBC, việc báo lỗi padding hoặc thời gian xử lý khác nhau tạo thành oracle; lặp lại truy vấn trên ciphertext sửa đổi cho phép suy ra plaintext.

Số truy vấn cho 1 KB. Với $N = 1024$ byte, tối đa 255 thử/byte:

$$Q \approx 255 \times N = 255 \times 1024 \approx 261,120 \text{ truy vấn.}$$

Con số này nhỏ với tấn công mạng hiện đại; trái lại brute-force là 255^{1024} (bất khả thi).

2.2.2 (b) (4 điểm) Đánh giá CCA và AE cho các cấu trúc

Yêu cầu: Đánh giá CCA/AE cho các cấu trúc và giải thích ngắn gọn; nêu thêm kịch bản replay và vai trò AD.

Trả lời:

Cấu trúc	CCA	AE	Giải thích
Encrypt-then-MAC	CÓ	CÓ	MAC trên ciphertext chặn chỉnh sửa trước khi giải mã; đạt AEAD tiêu chuẩn.
Encrypt-and-MAC	KHÔNG	KHÔNG	MAC trên plaintext rò rỉ lặp lại (tag trùng khi M trùng); không gắn kết với ciphertext.
MAC-then-encrypt	CÓ THỂ	CÓ THỂ	Phụ thuộc Enc; dễ lỗi do padding/timing; không khuyến nghị bằng EtM.

Replay và AD. Replay vẫn thành công nếu ngữ cảnh không được ràng buộc. Dữ liệu liên kết (AD) như ID phiên/timestamp/loại lệnh được đưa vào tính MAC; phát lại với AD cũ sẽ bị từ chối.

2.2.3 (c) (3 điểm) AES-GCM (Galois/Counter Mode)

Yêu cầu:

1. Mô tả kết hợp CTR với nhân trong $GF(2^{128})$ để xác thực (GHASH/GMAC) và lợi ích so với mã hóa+MAC rời.
2. Nêu tác động nghiêm trọng của tái sử dụng nonce.
3. Phân tích đánh đổi độ dài thẻ (128/64/32-bit).
4. Nêu một lỗ hổng triển khai đáng chú ý khác.

Trả lời:

CTR + GHASH. GCM dùng AES-CTR cho bí mật và GHASH/GMAC trên (IV, AD, C) cho xác thực $\Rightarrow$ AEAD gọn, hiệu năng cao.

Nonce tái sử dụng: thảm họa. Mất bí mật: nếu cùng nonce, keystream lặp lại, $C_1 \oplus C_2 = M_1 \oplus M_2$. Mất xác thực: có thể rên tag qua GHASH.

Độ dài thẻ. 128-bit: 2^{-128} (khuyến nghị). 64-bit: rủi ro tăng (birthday tầm 2^{32}). 32-bit: không an toàn.

Lỗi hỏng khác. Kênh bên thời gian/cache; cần AES-NI hoặc code constant-time.

3 Hàm Băm Chống Va Chạm (Collision-Resistant Hash Functions)

3.1 Câu 1. (15 điểm) Thuộc tính Hàm băm

3.1.1 (a) (5 điểm) Khả năng chống va chạm (Collision Resistance)

Yêu cầu:

1. Giải thích tại sao va chạm phải tồn tại trong bất kỳ hàm băm nào ánh xạ đầu vào có độ dài tùy ý sang đầu ra có độ dài cố định.
2. Sử dụng nghịch lý sinh nhật (birthday paradox), tính toán xấp xỉ số lượng băm phải được tính để tìm va chạm với xác suất 50% trong một hàm băm an toàn 256-bit.
3. Mô tả một kịch bản tấn công thực tế trong đó việc tìm va chạm băm sẽ làm tổn hại đến một hệ thống bảo mật.

Trả lời:

1) Sự tồn tại của va chạm. Va chạm chắc chắn tồn tại do nguyên lý lỗ bồ câu: ánh xạ từ tập vô hạn (đầu vào có độ dài tùy ý) sang tập hữu hạn (đầu ra có độ dài cố định). An ninh hàm băm là *va chạm không thể tìm thấy trong thực tế tính toán*, chứ không phải va chạm không tồn tại.

2) Tính toán birthday paradox. Với hàm băm 256-bit, số lượng băm cần thiết để tìm va chạm với xác suất 50%:

$$Q \approx \sqrt{2^{256}} = 2^{128}$$

Cần khoảng 2^{128} phép toán để tìm một va chạm.

3) Kịch bản tấn công thực tế (giả mạo tài liệu). Kẻ tấn công tạo hai tài liệu khác nhau $M_{\text{hợp pháp}}$ và $M_{\text{gian lận}}$ sao cho $H(M_{\text{hợp pháp}}) = H(M_{\text{gian lận}})$. Yêu cầu nạn nhân ký điện tử lên $H(M_{\text{hợp pháp}})$, sau đó gắn chữ ký vào $M_{\text{gian lận}}$, khiến nạn nhân bị ràng buộc với tài liệu gian lận.

3.1.2 (b) (5 điểm) Cấu trúc hàm băm

Yêu cầu:

1. So sánh và đối chiếu cấu trúc Merkle-Damgård (SHA-2) và cấu trúc Sponge (SHA-3).
2. Giải thích cách tấn công mở rộng độ dài hoạt động chống Merkle-Damgård và tại sao Sponge kháng được.
3. Mô tả cấu trúc HMAC và cách nó bảo vệ chống tấn công mở rộng độ dài.

Trả lời:

1) So sánh Merkle-Damgård và Sponge.

- **Merkle-Damgård (SHA-2):** Thông điệp được đệm và chia khối, xử lý tuần tự qua hàm nén. Dễ bị tấn công mở rộng độ dài.
- **Sponge (SHA-3):** Hoạt động qua hai pha: hấp thụ và vắt. Phần *capacity* của trạng thái nội bộ bị ẩn, chỉ phần *rate* được tiết lộ.

2) Tấn công mở rộng độ dài.

- **Merkle-Damgård:** Nếu biết $H(M)$ và độ dài M , có thể tính $H(M \parallel \text{padding} \parallel P)$ mà không cần biết nội dung M .
- **Sponge:** Kháng được vì phần *capacity* không được tiết lộ, không thể tiếp tục quá trình băm từ giá trị đã biết.

3) Cấu trúc HMAC.

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \| H((K \oplus \text{ipad}) \| m))$$

HMAC chống được tấn công mở rộng độ dài vì khóa K được dùng ở cả hai lớp băm, ngăn kẻ tấn công mở rộng thông điệp mà không biết khóa.

3.1.3 (c) (5 điểm) Sự phát triển của hàm băm

Yêu cầu:

1. Mô tả các tấn công thành công chống MD5 và SHA-1 dẫn đến việc chúng bị loại bỏ.
2. Giải thích khái niệm va chạm tiền tố đã chọn và tại sao chúng đặc biệt nguy hiểm với cơ quan cấp chứng chỉ.
3. So sánh bảo mật của SHA-2 và SHA-3 chống các kỹ thuật phân tích mật mã đã biết.

Trả lời:

1) Tấn công chống MD5 và SHA-1.

- **MD5 (128-bit):** Bị phá vỡ hoàn toàn từ năm 2004 (Xiaoyun Wang).
- **SHA-1 (160-bit):** Bị phá vỡ năm 2017 trong tấn công **SHAttered** của Google và CWI Amsterdam, yêu cầu sức mạnh tính toán tương đương 6,500 CPU-năm và 110 GPU-năm.

2) Va chạm tiền tố đã chọn. Cho hai tiền tố bất kỳ P_1, P_2 , tồn tại các hậu tố S_1, S_2 sao cho $H(P_1 \| S_1) = H(P_2 \| S_2)$. Đặc biệt nguy hiểm với cơ quan cấp chứng chỉ vì có thể giả mạo chứng chỉ hợp pháp.

3) So sánh SHA-2 và SHA-3.

- **SHA-2:** Sử dụng Merkle-Damgård, vẫn an toàn (256/512-bit), chưa có tấn công thực tế thành công.
- **SHA-3 (Keccak):** Sử dụng cấu trúc Sponge hoàn toàn mới, kháng các kỹ thuật đã phá MD5/SHA-1, cung cấp **Protocol Agility**.

3.2 Câu 2. (15 điểm) Băm mật khẩu (Password Hashing)

3.2.1 (a) (5 điểm) Phân tích lưu trữ mật khẩu

Yêu cầu: Phân tích tác động bảo mật nếu DB bị xâm phạm cho các cách lưu trữ: (1) plaintext, (2) mã hóa cùng khóa trên server, (3) SHA-256 không muối, (4) SHA-256 có muối, (5) Scrypt.

Trả lời:

#	Phương pháp lưu trữ	Tác động bảo mật	Phân tích
1	Plaintext	Thảm họa	Kẻ tấn công lấy ngay lập tức tất cả mật khẩu.
2	Mã hóa (khóa trên cùng server)	Thất bại	Kẻ tấn công lấy được cả khóa giải mã, làm thất bại mã hóa.
3	SHA-256 không muối	Kém	Dễ bị bảng rainbow/từ điển vì cùng mật khẩu $\Rightarrow$ cùng hash.
4	SHA-256 có muối	Tốt	Kháng rainbow, nhưng yếu nếu băm quá nhanh (brute-force dễ).
5	Scrypt/Argon2	Xuất sắc	Muối + memory-hard giảm lợi thế GPU/ ASIC.

Bảng 1: So sánh các phương pháp lưu trữ mật khẩu

3.2.2 (b) (5 điểm) Muối (Salting)

Yêu cầu:

1. Muối chống bảng rainbow như thế nào.
2. Tính dung lượng lưu trữ với 10,000 người dùng, muối 16B, hash 32B.
3. Thực hành tốt nhất tạo/lưu trữ muối.

Trả lời:

#	Phương pháp lưu trữ	Tác động bảo mật	Phân tích
1	Plaintext	Thảm họa	Kẻ tấn công lấy ngay lập tức tất cả mật khẩu.
2	Mã hóa (khóa trên cùng server)	Thất bại	Kẻ tấn công lấy được cả khóa giải mã, làm thất bại mã hóa.
3	SHA-256 không muối	Kém	Dễ bị bảng rainbow/từ điển vì cùng mật khẩu $\Rightarrow$ cùng hash.
4	SHA-256 có muối	Tốt	Kháng rainbow, nhưng yếu nếu băm quá nhanh (brute-force dễ).
5	Scrypt/Argon2	Xuất sắc	Muối + memory-hard giảm lợi thế GPU/ ASIC.

Bảng 2: So sánh các phương pháp lưu trữ mật khẩu

3.2.3 (c) (5 điểm) Hàm băm mật khẩu chuyên dụng

Yêu cầu: So sánh memory-hard (Scrypt/Argon2) với PBKDF2; giải thích tham số Scrypt (N, r, p) ; so sánh tốc độ SHA-256, PBKDF2, Scrypt và ý nghĩa bảo mật.

Trả lời:

Memory-hard vs PBKDF2. PBKDF2 tăng CPU qua lặp PRF; Scrypt/Argon2 buộc dùng bộ nhớ lớn $\Rightarrow$ giảm lợi thế GPU/ ASIC.

Tham số Scrypt.

- N (CPU cost): Tăng số vòng lặp, tăng thời gian tính toán.
- r (block size): Tăng bằng thông/bộ nhớ.
- p (parallelization): Tăng song song hóa và chi phí tổng.

Tốc độ và tác động.

SHA-256 $\gg$ PBKDF2 $\gg$ Scrypt/Argon2

GPU hiện đại $> 2 \times 10^9$ SHA-256/s, nhưng chỉ vài nghìn Scrypt/Argon2/s $\Rightarrow$ chi phí vét cạn tăng mạnh, làm tấn công ngoại tuyến kém khả thi.

4 Nghiên Cứu Diễn Hình Mật Mã Ứng Dụng

4.1 Câu 1. (10 điểm) Phân Tích Các Chế Độ Mã Khối

Yêu cầu: Với tham chiếu đến các chế độ mã khối được đề cập trong bài giảng, phân tích các kịch bản sau:

4.1.1 (a) (4 điểm) Ứng dụng lưu trữ tệp an toàn (Tệp lớn)

Yêu cầu: Một ứng dụng lưu trữ tệp an toàn cần mã hóa các tệp người dùng. So sánh các chế độ CBC, CTR và AES-GCM cho ứng dụng này, thảo luận về:

1. Tác động hiệu suất đối với các tệp lớn.
2. Lan truyền lỗi nếu các phần của văn bản mã hóa bị hỏng.
3. Các tác động bảo mật của việc tái sử dụng IV/nonce.
4. Đảm bảo tính toàn vẹn dữ liệu và lợi thế của mã hóa xác thực với AES-GCM.

Trả lời:

Tiêu chí	CBC	CTR	AES-GCM
Hiệu suất (Tệp lớn)	Kém: mã hóa tuần tự, không song song hóa.	Tốt nhất: mã hóa/giải mã hoàn toàn song song.	Tốt nhất: song parallel nhờ CTR + GHASH.
Lan truyền lỗi	Lỗi khối $C_i \Rightarrow$ ảnh hưởng M_i và M_{i+1} .	Lỗi chỉ ảnh hưởng khối bị hỏng (M_i).	Lỗi bất kỳ $\Rightarrow$ Tag mismatch, từ chối toàn bộ.
Tái sử dụng IV/Nonce	IV tái sử dụng $\Rightarrow$ rò rỉ lặp plaintext đầu tiên.	Nonce tái sử dụng $\Rightarrow$ keystream lặp, mất bí mật.	Nonce tái sử dụng $\Rightarrow$ mất bí mật & xác thực.
Tính toàn vẹn	Không (cần MAC riêng).	Không (cần MAC riêng).	Có (GHASH xác thực tích hợp).
Kết luận	Tránh dùng	Tốt nếu không cần xác thực	Tốt nhất: Hiệu suất + Xác thực

Bảng 3: So sánh CBC, CTR, AES-GCM cho lưu trữ tệp lớn

Lợi thế của AES-GCM: AES-GCM là lựa chọn tối ưu vì kết hợp:

- **Hiệu suất cao:** Tận dụng song parallel hóa của CTR + GHASH.
- **Mã hóa Xác thực (AEAD):** Đảm bảo tệp không bị thao túng trong lưu trữ.
- **Phát hiện lỗi:** Bất kỳ sai sót nào trong ciphertext đều bị phát hiện qua xác thực Tag.

4.1.2 (b) (3 điểm) Ứng dụng nhắn tin thời gian thực (Tin nhắn ngắn)

Yêu cầu: Một ứng dụng nhắn tin thời gian thực cần mã hóa các tin nhắn ngắn với độ trễ tối thiểu. So sánh các chế độ CBC, CTR và AES-GCM cho ứng dụng này, thảo luận về:

1. Khả năng song parallel hóa cho mã hóa/giải mã.
2. Tính phù hợp cho dữ liệu phát trực tuyến (streaming).
3. Bảo vệ chống lại các cuộc tấn công văn bản mã hóa đã chọn.
4. Cách AES-GCM giải quyết nhu cầu xác thực so với các chế độ không xác thực.

Trả lời:

1) **Khả năng song parallel hóa.** CBC mã hóa tuần tự (không song parallel); CTR và AES-GCM mã hóa hoàn toàn song parallel, phù hợp với tin nhắn ngắn/độ trễ thấp.

2) **Tính phù hợp cho streaming.** CTR hoạt động như một mã dòng (keystream độc lập); AES-GCM cũng song parallel nhưng cần toàn bộ dữ liệu để tính Tag cuối. Cả hai đều phù hợp hơn CBC.

3) **Bảo vệ chống CCA.** CBC và CTR dễ bị Padding/Null Oracle; AES-GCM an toàn CCA nhờ xác thực Tag trước khi giải mã.

4) **Giải quyết nhu cầu xác thực.** Chỉ AES-GCM cung cấp xác thực tích hợp (GHASH). CBC, CTR cần HMAC riêng, tăng chi phí.

Kết luận: AES-GCM tốt nhất cho ứng dụng nhắn tin, vừa có hiệu suất cao vừa đảm bảo xác thực thông điệp.

4.1.3 (c) (3 điểm) Cụ thể đối với AES-GCM

Yêu cầu: Giải thích tác động bảo mật của việc tái sử dụng nonce trong AES-GCM so với CTR. Thảo luận về sự đánh đổi hiệu suất AES-GCM vs CTR+HMAC. Giải thích cách AEAD bảo vệ chống tấn công CCA.

Trả lời:

1) Tái sử dụng nonce: AES-GCM vs CTR.

- **CTR:** Nonce tái sử dụng $\Rightarrow$ keystream $G(K, N)$ lặp lại. Kẻ tấn công XOR hai ciphertext: $C_1 \oplus C_2 = M_1 \oplus M_2$. **Mất bí mật.**
- **AES-GCM:** Nonce tái sử dụng $\Rightarrow$ keystream lặp (mất bí mật) + GHASH không phục hồi được, cho phép giả mạo Tag. **Mất bí mật + xác thực.**

2) Hiệu suất: AES-GCM vs CTR+HMAC.

- **AES-GCM:** GHASH song parallel, tận dụng AES-NI/PCLMULQDQ, kết hợp mã hóa + xác thực trong một lược đồ.
- **CTR+HMAC:** Gọi hai primitive riêng, overhead lớn hơn.
- **Kết luận:** AES-GCM hiệu suất tốt hơn CTR+HMAC.

3) Bảo vệ chống tấn công CCA.

- **CBC/CTR:** Dẻo dai (malleable), kẻ tấn công sửa ciphertext, khai thác Oracle giải mã để học plaintext.
- **AEAD (AES-GCM):** Áp dụng "Verify before Decrypt". Bất kỳ thao túng ciphertext nào $\Rightarrow$ Tag mismatch, từ chối toàn bộ trước khi giải mã. Ngăn chặn truy vấn Oracle thành công.
- **Kết luận:** AEAD an toàn CCA, CBC/CTR không.

4.2 Câu 2. (10 điểm) Phân Tích Bảo Mật Hàm Băm

Mô tả hệ thống: Một hệ thống cập nhật phần mềm sử dụng hàm băm để xác minh tính toàn vẹn:

1. Nhà cung cấp đăng các băm SHA-1 của các tệp cập nhật trên trang web HTTPS.
2. Người dùng tải xuống tệp qua HTTP (tiết kiệm băng thông).
3. Ứng dụng xác minh tệp bằng cách tính băm SHA-1 và so sánh với băm từ HTTPS.
4. Nếu khớp, bản cập nhật được cài đặt tự động.

4.2.1 (a) Xác định ít nhất ba lỗ hổng bảo mật

Trả lời:

1. **Sử dụng SHA-1:** SHA-1 đã bị phá vỡ (tấn công SHAttered). Lỗ hổng: Va chạm (Collision).
2. **Tải xuống tệp qua HTTP:** Kênh không xác thực, không bảo vệ tính toàn vẹn. Lỗ hổng: Tấn công Man-in-the-Middle (MITM).
3. **Thiếu Xác thực Người gửi:** Chỉ xác minh tính toàn vẹn, không có MAC hoặc Chữ ký số. Lỗ hổng: Giả mạo (Impersonation).

4.2.2 (b) Kịch bản tấn công cụ thể

Trả lời:

1) **Tấn công Va chạm SHA-1.** Kẻ tấn công tạo hai tệp $M_{\text{hợp pháp}}$ và $M_{\text{độc hại}}$ sao cho $H_{\text{SHA-1}}(M_{\text{hợp pháp}}) = H_{\text{SHA-1}}(M_{\text{độc hại}})$. Chặn HTTP, thay thế $M_{\text{hợp pháp}}$ bằng $M_{\text{độc hại}}$. Ứng dụng cài đặt tệp độc hại vì băm khớp.

2) **Tấn công MITM / Downgrade.** Kẻ tấn công chặn HTTP, thay tệp mới bằng phiên bản cũ (có lỗ hổng). Nếu băm SHA-1 của phiên bản cũ vẫn trong danh sách được công bố, ứng dụng chấp nhận và cài đặt phiên bản dễ bị tấn công.

3) **Giả mạo Tệp và Băm.** Nếu kẻ tấn công làm hỏng HTTPS/DNS, họ có thể đăng băm độc hại trên trang web giả mạo. Vì thiếu chữ ký số, người dùng không thể phân biệt.

4.2.3 (c) Đề xuất cải tiến

Trả lời:

Lỗ hổng	Đề xuất Cải tiến	Duy trì Hiệu suất
Sử dụng SHA-1	Nâng cấp lên SHA-256 hoặc SHA-3	SHA-256 rất nhanh
Thiếu Xác thực	Ký điện tử vào băm (ECDSA/RSA)	Ký 1 lần trên hash nhỏ
Tải qua HTTP	Giữ tệp qua HTTP, tắt cài tự động	Buộc xác minh chữ ký trước

Bảng 4: Cải tiến hệ thống cập nhật

4.2.4 (d) Thiết kế hệ thống thay thế an toàn

Trả lời:

Kỹ thuật: Mã hóa Xác thực (AEAD) + Mật mã Khóa Công khai (PKC).

Quy trình Bên cung cấp:

1. Tệp cập nhật (M).
2. Tính băm $H_{\text{SHA-256}}(M)$.
3. Ký số: $\text{Sig} \leftarrow \text{PKC}.\text{Sign}(K_{\text{PR}}, H(M))$.
4. Công bố: Tệp M , băm, chữ ký trên HTTPS.

Quy trình Bên người dùng:

1. Tải tệp M qua HTTP.
2. Tải băm $H(M)$ và Sig qua HTTPS.
3. **Bước 1 (Xác thực Người gửi):** Xác minh Sig qua khóa công khai nhà cung cấp.

4. **Bước 2 (Toàn vẹn):** Tính $H_{\text{SHA-256}}(M)$ đã tải, so sánh với $H(M)$ xác minh.
5. **Bước 3:** Nếu cả hai thành công, cài đặt.

Lợi thế:

- **Bảo mật cao:** Chữ ký số $\Rightarrow$ xác thực người gửi.
- **Hiệu suất:** Ký/xác minh 1 lần trên hash nhỏ.
- **Linh hoạt:** Tập lớn qua HTTP, hash/chữ ký qua HTTPS.

4.3 Câu 3. (10 điểm) Thiết Kế Hệ Thống Quản Lý Mật Khẩu

Yêu cầu hệ thống:

- Khôi phục tài khoản an toàn khi quên mật khẩu.
- Kháng tấn công từ điển ngoại tuyến nếu DB bị xâm phạm.
- Hỗ trợ xác thực hiệu suất cao cho cơ sở người dùng lớn.
- Phát hiện và ngăn chặn credential stuffing.

4.3.1 (a) Nguyên thủy mật mã để lưu trữ mật khẩu

Trả lời:

Nguyên thủy chính: Hàm băm mật khẩu chuyên dụng Memory-Hard như **Argon2** (hoặc Scrypt).

Lưu trữ:

- Hàm băm của mật khẩu.
- Salt ngẫu nhiên duy nhất (≥ 16 byte).
- Tham số chi phí (N, r, p cho Argon2).

Tại sao:

1. **Kháng tấn công từ điển ngoại tuyến:** Salt vô hiệu hóa Bảng Rainbow.
2. **Kháng tấn công vét cạn:** Tính Memory-Hard tăng chi phí GPU/ASIC lên đáng kể.

4.3.2 (b) Cơ chế khôi phục mật khẩu

Trả lời:

Cơ chế: Password Reset Token qua kênh email/SMS.

Quy trình:

1. Người dùng yêu cầu đặt lại mật khẩu.
2. Hệ thống tạo Token ngẫu nhiên, bí mật, thời gian giới hạn (ví dụ: 128 bit).
3. Lưu trữ: Băm SHA-256 của Token + Thời gian hết hạn trong DB (không phải Token plaintext).
4. Gửi Token (plaintext) qua email.
5. Người dùng nhập liên kết, hệ thống băm Token nhận được, so sánh với băm DB.
6. Nếu khớp + chưa hết hạn, cho phép đặt mật khẩu mới.

Phân tích Bảo mật:

- Chỉ lưu băm Token $\Rightarrow$ ngay cả khi DB xâm phạm, kẻ tấn công không sử dụng được Token.
- An ninh phụ thuộc vào: tính bí mật + thời gian giới hạn Token.
- Chuyển gánh nặng bảo mật sang kênh email người dùng.

4.3.3 (c) Cân bằng Bảo mật vs Hiệu suất

Trả lời:

1) **Bảo mật ưu tiên:** Tham số chi phí cao cho Argon2 (ví dụ: thời gian băm $\approx 500\text{ms}$).

2) **Cân bằng Hiệu suất:**

- **Xác thực:** Quá trình băm chậm chỉ xảy ra 1 lần/đăng nhập. Độ trễ 500ms chấp nhận được.
- **Tải lớn:** Argon2 hỗ trợ song song parallel (tham số p) để phân tán tải băm trên nhiều lõi CPU $\Rightarrow$ hỗ trợ cơ sở người dùng lớn.

4.3.4 (d) Lỗ hổng tiềm ẩn và giảm thiểu

Trả lời:

Lỗ hổng	Giảm thiểu (Mitigation)
Credential Stuffing	Theo dõi tần suất đăng nhập thất bại. Triển khai MFA bắt buộc.
Timing Side-Channel	So sánh hàm băm bằng constant-time comparison.
Token bị đánh cắp	TTL ngắn (15 phút), one-time use, đăng xuất toàn bộ thiết bị.

Bảng 5: Lỗ hổng tiềm ẩn và cách giảm thiểu